
Master Theorem Simplified

INF1008 Data Structures and Algorithms

Name: Timothy Chia

Student ID: 2501530

The Master Theorem. Divide-and-conquer algorithms are commonly analysed using recurrence relations. A large class of such recurrences can be solved using the *Master Theorem*, which provides asymptotic bounds without requiring full expansion of the recurrence.

Problem Form

The Master Theorem applies to recurrences of the form:

$$T(n) = \begin{cases} \Theta(1), & n < c_1 \\ aT\left(\frac{n}{b}\right) + \Theta(n^i), & n \geq c_1 \end{cases}$$

where:

- $a \geq 1$ is the number of subproblems,
- $b > 1$ is the factor by which the input size is reduced,
- $\Theta(n^i)$ represents the non-recursive work done at each level,
- c_1 is a constant defining the base case threshold.

This recurrence models algorithms that split a problem into smaller subproblems, solve them recursively, and perform additional work to combine the results.

Recursion Tree Interpretation

The recurrence can be visualised as a recursion tree:

- The height of the tree is $\log_b n$.
- Each level contains a^k subproblems of size n/b^k .
- The total work at each level depends on both the number of subproblems and the cost per subproblem.

The Master Theorem determines where the dominant contribution to the total runtime occurs: at the root, evenly across all levels, or at the leaves.

Asymptotic Cases

Let b^i represent the growth rate of the non-recursive work per level.

Case 1: $a > b^i$ (Leaf-Dominated)

If the number of subproblems grows faster than the work per level:

$$T(n) = \Theta(n^{\log_b a})$$

In this case, the total work increases as the recursion proceeds downward, and the leaves of the recursion tree dominate the overall complexity.

Case 2: $a = b^i$ (Balanced Contribution)

If the recursive and non-recursive work grow at the same rate:

$$T(n) = \Theta(n^i \log n)$$

Here, each level of the recursion tree contributes an equal amount of work. The total runtime is the product of the work per level and the height of the tree.

Case 3: $a < b^i$ (Root-Dominated)

If the non-recursive work dominates:

$$T(n) = \Theta(n^i)$$

The majority of the work is performed near the root of the recursion tree, and the cost of deeper recursive calls becomes negligible.

Summary

This simplified definition provides a fast and reliable method for analysing recurrences by comparing the growth rate of recursive calls (a) against the growth rate of non-recursive work (b^i). By identifying which part of the recursion tree dominates the computation, the asymptotic time complexity can be determined directly.